Breakfast Club
產品設計與研發案例整理

產品名稱	Breakfast Club
產品型態	LINE Bot + Web App + Supabase + Gemini
文件用途	作品集參考檔案 / Case Study 原稿
整理依據	兩份技術交接文件與狀態報告整併改寫

摘要｜Breakfast Club 的產品核心不是「記錄早餐」本身，而是把一個原本私密、容易中斷的習慣，轉化為可被看見、可被互動、可被回饋的社交行為。整體研發過程也因此不只是功能堆疊，而是不斷在「使用者願不願意每天打開」與「架構能不能撐得住持續成長」之間做取捨。

一、產品發想：從個人習慣追蹤，到具社交動能的早餐平台

如果只把 Breakfast Club 當作一個早餐打卡工具，它其實很容易落入同質化：拍照、記錄、看熱量，然後使用者在新鮮感過後自然流失。真正讓這個產品成立的，是我們把「早餐」定義成一個每天都會發生、但通常不會被分享的行為錨點，進一步用社交可見性去放大它的持續性。

產品假設建立在一個很直接的行為洞察上：當一件原本私人的日常行為被放入帶有觀眾感的情境，完成率與留存機率通常會提高。因此，Breakfast Club 並不是要幫使用者做更精密的營養管理，而是先用「看得見的打卡、可互動的早餐牆、排行榜與揪團」把日常習慣變成一個輕量但有黏性的社交循環。

這也決定了設計邏輯必須同時滿足三件事：

- 行為門檻要低：使用者最好只要拍一張照，就能完成一次完整互動。
- 回饋速度要快：辨識、鼓勵、展示與社交反應都要足夠即時，才有「我今天真的完成了一件事」的感受。
- 視覺與語氣要有記憶點：產品需要一個輕鬆但清楚的世界觀，讓打卡這件事不是冷冰冰的紀錄，而像加入一個每天早上都會亮起來的小社群。

產品定位與介面語言

在品牌與體驗層面，我們刻意採用 1950s 美式 diner 的視覺語言，讓產品從一開始就不是健康管理工具的典型模板，而是比較有情緒、有記憶點的早餐俱樂部。個人頁被命名為 Receipt，社群牆叫 The Wall，揪團功能叫 Neon Meetup，這些命名都不是裝飾，而是在替整個系統建立一致的使用情境。

二、產品設計邏輯：為什麼採用 LINE Bot + Web App 的雙入口架構

從產品策略上，我們沒有直接把所有流程都塞進單一 App，而是採用雙入口設計：LINE Bot 處理最低摩擦的每日打卡，Web App 承接記錄回顧、社群互動、排行榜與揪團等需要視覺化和多步操作的情境。

- LINE Bot 是高頻入口：使用者最容易在原本就會打開的通訊場景裡完成拍照與回覆。
- Web App 是內容與互動場：需要瀏覽牆面、查看個人紀錄、留言、參加揪團時，Web 介面更能承載資訊密度。
- 雙入口讓我們可以先驗證行為是否成立，再逐步補足社群與資料體驗，而不是一開始就做一個成本高、啟動門檻也高的完整 App。

技術選型也反映了這個邏輯：後端以 Flask 承接 LINE Webhook 與高風險寫入，Web 前端使用 React SPA 承接互動頁面，Supabase 負責資料儲存與政策控管，Gemini Vision 負責早餐辨識與熱量估算，ImgBB 則作為輕量圖床，以降低初期儲存成本與部署複雜度。

核心架構總覽

模組	設計目的	關鍵角色
LINE Bot	降低每日打卡摩擦	接收圖片、觸發 Gemini、更新 streak
React Web App	承接視覺化與社群互動	社群牆、個人頁、排行榜、揪團
Flask Backend	承接高風險寫入與邏輯整合	like/save/visibility/profile API
Supabase	資料持久化與權限控管	Postgres、RLS、查詢與互動資料
Gemini + ImgBB	完成辨識與媒體處理	食物辨識、熱量估算、圖片外部存放

三、研發歷程：從功能可用，到權限可控、資料一致、互動穩定

如果從產品開發者的角度回看，Breakfast Club 的研發歷程其實很典型：前期先用可驗證產品價值的方式快速做出互動閉環，等到功能真的開始變多、資料表之間的依賴開始複雜，安全性與資料一致性的問題才會真正浮出來。這也是為什麼後續技術交接文件的核心，不再只是功能說明，而是大量聚焦在 RLS、schema 對齊、寫入路徑與前後端責任分工。

第一階段：先讓核心循環跑起來

第一階段的目標很單純：讓使用者可以透過 LINE 傳早餐照片，得到辨識結果與鼓勵回覆，並在 Web 端看到個人紀錄與社群內容。此時最重要的是打通完整閉環，而不是一開始就把所有資料治理做滿。

- 使用者在 LINE 傳圖後，系統會在台北時間 04:00-10:00 的打卡視窗內判斷是否為有效早餐。
- Gemini Vision 回傳結構化 JSON，包含是否為早餐、食物名稱、熱量估算、鼓勵短評與可能地點。
- 若符合規則，就更新連續打卡天數並建立貼文；若不符合，也仍可保留紀錄但不計入 streak。
- Web 端則延伸出 Receipt、Wall、Meet、Rank 等體驗，讓產品不只停留在「有沒有打卡」，而開始有被看見與被互動的理由。

第二階段：當互動功能增加，權限與資料一致性問題開始出現

隨著 like、save、留言、公開切換與揪團功能逐步增加，原本由前端直接寫入資料庫的模式開始碰到兩個典型問題：第一，RLS 一旦啟用，原本能動的功能會突然被擋；第二，資料表 schema 與文件假設不一致時，系統會出現難追的 500 錯誤。

這個階段最關鍵的設計決策，是把高風險寫入逐步搬到 backend。這不只是安全優化，更是產品可持續迭代的前提。當使用者互動越來越多，產品團隊需要的是一條可控、可追蹤、能持續擴充的寫入管線。

第三階段：進入 Hybrid 過渡期，重新劃分前後端責任

後續系統進入一個很典型的 Hybrid 過渡期：前端仍直接讀取多張表，但高風險寫入已逐步改由 Flask 後端承接。這代表團隊不是一次性重構，而是依風險高低分批搬遷，優先把會被 RLS 擋下、或會造成資料污染的操作收斂到 backend。

- 已搬到 backend 的寫入：改暱稱、貼文公開切換、like、save。
- 仍留在前端的寫入：發文、留言、發起活動、加入活動，以及部分 dev 操作。
- 這樣的取捨讓產品可以在不全面停機重寫的情況下，先穩住最常用、最容易出問題的互動。

四、研發過程中碰到的代表性問題，以及我是如何理解並解決它們

1. Like / Save 功能寫入成功，重新整理後卻消失

這類問題最容易被誤判成後端寫失敗，但實際根因是讀取權限沒有補齊。系統後來確認，like/save 寫入本身其實已成功，只是 social_actions 缺少前端 SELECT policy，導致 reload 後前端讀不到剛剛的互動狀態，看起來像資料消失。

這件事對產品與工程兩邊都很有提醒性：有時候使用者感受到的是「功能不穩」，但真正的問題不是單點 bug，而是資料生命周期中的某一段沒有對齊。解法不是只 patch 前端畫面，而是補齊 social_actions 的讀取 policy，讓寫入後的狀態能被正確重建。

2. social_actions 的 schema 與文件假設不一致，導致 500 錯誤

原先文件假設使用的是 user_id / post_id 欄位，但實際 schema 是 actor_id / target_post_id / action_type。這種問題表面上像單純欄位命名差異，實際上反映的是產品開發常見的風險：當資料模型沒有被明確管理，開發文件、前端邏輯與後端實作會逐漸漂移。

解法不是只改一個查詢，而是把前後端對同一張表的欄位認知全部拉齊，並同步修正文檔。這種校準工作雖然不顯眼，但它直接決定後面能不能放心接更多功能。

3. Like 與 Save 互相衝突

social_actions 舊的唯一約束只鎖 target_post_id 與 actor_id，沒有把 action_type 納入，因此同一個人對同一篇貼文按 like 與 save 時，兩種互動會互相卡住。這是很典型的「產品語意正確，但資料約束不夠精確」的問題。

解法是把唯一約束調整為包含 action_type，讓 like 與 save 可以被視為同一位使用者對同一篇內容的兩種不同互動。這個修正不只是資料庫層級的 patch，也是在重新定義產品互動模型。

4. 活動列表消失，真正原因不是資料遺失，而是依賴表不存在

活動頁的列表一度整體消失，直覺上很容易懷疑 meetup_events 資料表或查詢壞掉，但後來確認真正根因是 meetup_attendees 表根本不存在。由於活動列表在重建過程中依賴這張表計算參加人數，少掉依賴表後整個頁面就一起失效。

這個案例很值得放進作品集，因為它能展現你不是只停留在畫面層除錯，而是會回頭看資料依賴鏈。最終解法是補建 meetup_attendees、加上唯一約束與 SELECT policy，讓活動資料與參加人數都能正常 reload。

5. fetchAll() 單一大 try/catch 讓任何一張表出錯都拖垮整頁

這是典型的前端資料協調技術債。當多張表都在同一個 fetchAll() 裡，只要其中一張表查詢失敗，整體畫面重建就可能一起受影響。從產品角度看，這種錯誤會讓使用者覺得系統「常常整塊壞掉」，而不是某個子功能暫時不可用。

因此後續建議不是再補更多例外判斷，而是把資料拉取拆段，各自處理錯誤與回退策略。這是讓產品從 prototype 走向穩定服務必須補上的工程紀律。

五、下一步若要繼續深化，我會怎麼規劃

- 先完成發文、留言、發起活動、join meetup 等路徑的 API 化，結束前端直寫資料庫的過渡期。
- 在身份模型穩定後，再收緊 RLS，避免一開始就因權限過嚴造成整站功能癱瘓。

- 將好友系統與 Realtime 更新視為下一階段核心槓桿，因為這會讓產品真正從打卡工具升級成有社交動能的平台。
- 補強資料讀取層的容錯與分段載入，降低單點失敗拖垮整個畫面的風險。

Breakfast Club 是一個以 LINE Bot 為低摩擦入口、Web App 為互動與社群場景的早餐習慣產品。我從產品設計邏輯、互動閉環到後續權限與資料治理都持續參與：前期聚焦在驗證「早餐打卡 + 社交可見性」是否成立，後期則逐步把高風險寫入從前端遷移到後端，處理 RLS、schema 漂移、互動資料衝突與依賴表缺失等問題，讓產品從可用走向可維運。